

tf.compat.v1.losses.mean_squared_error

Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/losses/mean_squared_error

Adds a Sum-of-Squares loss to the training procedure.

Function Signatures

```
tf.compat.v1.losses.mean_squared_error( labels, predictions,  
weights=1.0, scope=None,  
loss_collection=ops.GraphKeys.LOSSES,  
reduction=Reduction.SUM_BY_NONZERO_WEIGHTS )
```

Code Examples

```
tf.compat.v1.losses.mean_squared_error(  
labels,  
predictions,  
weights=1.0,  
scope=None,  
loss_collection=ops.GraphKeys.LOSSES,  
reduction=Reduction.SUM_BY_NONZERO_WEIGHTS  
)
```

```
tf.compat.v1.losses.mean_squared_error(  
    labels,  
    predictions,  
    weights=1.0,  
    scope=None,  
    loss_collection=ops.GraphKeys.LOSSES,  
    reduction=Reduction.SUM_BY_NONZERO_WEIGHTS  
)
```

```
loss = tf.compat.v1.losses.mean_squared_error(  
    labels=labels,  
    predictions=predictions,  
    weights=weights,  
    reduction=reduction)
```

```
loss = tf.compat.v1.losses.mean_squared_error(  
    labels=labels,  
    predictions=predictions,  
    weights=weights,  
    reduction=reduction)
```

```
loss_fn = tf.keras.losses.MeanSquaredError(  
    reduction=reduction)  
loss = loss_fn(  
    y_true=labels,  
    y_pred=predictions,  
    sample_weight=weights)
```

```
loss_fn = tf.keras.losses.MeanSquaredError(  
    reduction=reduction)  
loss = loss_fn(  
    y_true=labels,  
    y_pred=predictions,  
    sample_weight=weights)
```

```
y_true = [1, 2, 3]
y_pred = [1, 3, 5]
weights = [0, 1, 0.25]
# samples with zero-weight are excluded from calculation when
`reduction`
# argument is set to default value
`Reduction.SUM_BY_NONZERO_WEIGHTS`
tf.compat.v1.losses.mean_squared_error(
labels=y_true,
predictions=y_pred,
weights=weights).numpy()
1.0
```

```
y_true = [1, 2, 3]
```

Documentation

Adds a Sum-of-Squares loss to the training procedure.

Migrate to TF2

`tf.compat.v1.losses.mean_squared_error` is mostly compatible with eager execution and `tf.function`. But, the `loss_collection` argument is ignored when executing eagerly and no loss will be written to the loss collections. You will need to either hold on to the return value manually or rely on `tf.keras.Model` loss tracking.

To switch to native TF2 style, instantiate the `tf.keras.losses.MeanSquaredError` class and call the object instead.

Structural Mapping to Native TF2

Before:

After:

How to Map Arguments

Before & After Usage Example

Before:

After:

Description

Used in the notebooks

- Migrate LoggingTensorHook and StopAtStepHook to Keras callbacks
- Migrate from Estimator to Keras APIs
- Migrate single-worker multiple-GPU training
- Migrate multi-worker CPU/GPU training
- Migrate SessionRunHook to Keras callbacks

weights acts as a coefficient for the loss. If a scalar is provided, then the loss is simply scaled by the given value. If weights is a tensor of size [batch_size], then the total loss for each sample of the batch is rescaled by the corresponding element in the weights vector. If the shape of weights matches the shape of predictions, then the loss of each measurable element of predictions is scaled by the corresponding value of weights.

Returns

Raises

